Лабораторна робота № 4



Студента групи КН-315

Лозового О.А.

```
!pip install scikit-surprise
```

```
Requirement already satisfied: scikit-surprise in /usr/local/lib/python3.10/dist-packages (1.1.3)
Requirement already satisfied: joblib>=1.0.0 in /usr/local/lib/python3.10/dist-packages (from scikit-surprise) (1.4.2)
Requirement already satisfied: numpy>=1.17.3 in /usr/local/lib/python3.10/dist-packages (from scikit-surprise) (1.25.2)
Requirement already satisfied: scipy>=1.3.2 in /usr/local/lib/python3.10/dist-packages (from scikit-surprise) (1.11.4)
```

```
import pandas as pd
import numpy as np
import ast

from sklearn.neighbors import NearestNeighbors
from sklearn.metrics.pairwise import cosine_similarity, cosine_distances


import matplotlib.pyplot as plt
import seaborn as sns

import warnings
warnings.filterwarnings("ignore")

from collections import defaultdict
from surprise import Dataset, Reader, KNNWithMeans, accuracy
from surprise.model_selection import GridSearchCV
from surprise.model_selection import train_test_split
from surprise.model_selection import KFold

from surprise import SVDpp


data = Dataset.load_builtin('ml-1m')
data
```

```
<surprise.dataset.DatasetAutoFolds at 0x7f54c901f220>
```

```
df = pd.DataFrame(data.raw_ratings, columns=["user_id", "item_id", "rating", "timestamp"])
df.count()
```

```
user_id      1000209
item_id      1000209
rating       1000209
```

```
timestamp    1000209
dtype: int64
```

```
df.head()
```

	user_id	item_id	rating	timestamp
0	1	1193	5.0	978300760
1	1	661	3.0	978302109
2	1	914	3.0	978301968
3	1	3408	4.0	978300275
4	1	2355	5.0	978824291

```
print('No. of Unique Users    :', df.user_id.nunique())
print('No. of Unique Movies :', df.item_id.nunique())
print('No. of Unique Ratings  :', df.rating.nunique())
```

```
No. of Unique Users    : 6040
No. of Unique Movies : 3706
No. of Unique Ratings  : 5
```

```
trainset, testset = train_test_split(data, test_size=0.2)
reader = Reader(rating_scale=(0, 5))
```

```
algo = SVDpp(cache_ratings=True)
algo.fit(trainset)
predictions0 = algo.test(testset)
rmse_score = accuracy.rmse(predictions0)
mae_score = accuracy.mae(predictions0)
```

```
RMSE: 0.8627
MAE:  0.6728
```

```
algo = SVDpp(**{'n_epochs':10, 'lr_all':0.01, 'reg_all': 0.05},cache_ratings=True)
algo.fit(trainset)
predictions1 = algo.test(testset)
rmse_score = accuracy.rmse(predictions1)
mae_score = accuracy.mae(predictions1)
```

```
RMSE: 0.8751
MAE:  0.6904
```

```
algo = SVDpp(**{'n_epochs':20, 'lr_all':0.01, 'reg_all': 0.05},cache_ratings=True)
algo.fit(trainset)
predictions2 = algo.test(testset)
rmse_score = accuracy.rmse(predictions2)
mae_score = accuracy.mae(predictions2)
```

```
RMSE: 0.8576
MAE:  0.6741
```

```
algo = SVDpp(**{'n_epochs':10, 'lr_all':0.001, 'reg_all': 0.05},cache_ratings=True)
algo.fit(trainset)
predictions3 = algo.test(testset)
rmse_score = accuracy.rmse(predictions3)
mae_score = accuracy.mae(predictions3)
```

```
RMSE: 0.9258
MAE:  0.7358
```

```
algo = SVDpp(**{'n_epochs':20, 'lr_all':0.001, 'reg_all': 0.05},cache_ratings=True)
algo.fit(trainset)
predictions4 = algo.test(testset)
rmse_score = accuracy.rmse(predictions4)
mae_score = accuracy.mae(predictions4)
```

```
RMSE: 0.9138
MAE:  0.7250
```

```
# повертає кількість елементів, оцінених даним користувачем
def get_num_user_ratings(uid):
    try:
        return len(trainset.ur[trainset.to_inner_uid(uid)])
    except ValueError:
        return 0
# повертає кількість користувачів, які оцінили даний товар
def get_num_item_ratings(iid):
    try:
        return len(trainset.ir[trainset.to_inner_iid(iid)])
    except ValueError:
        return 0

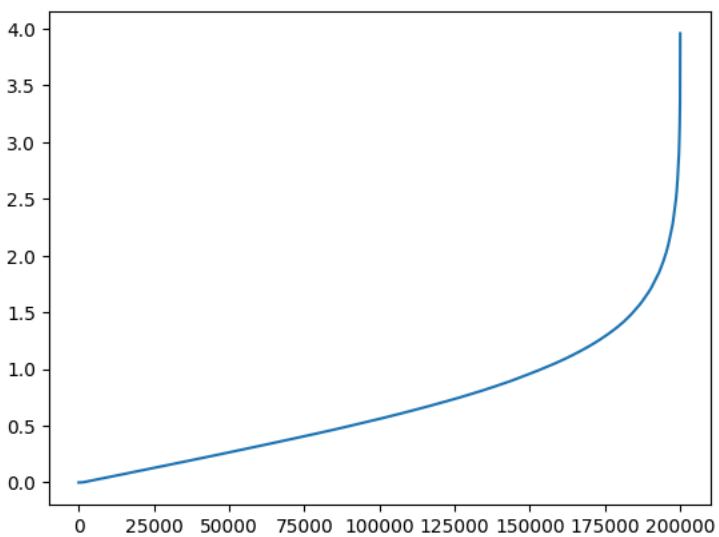

# для кожного прогнозу показані: хто (користувач), що (предмет), яка була реальна оцінка, яку модель прогнозувала. кількість
trainset = algo.trainset
predictions_df = pd.DataFrame(predictions2, columns=['uid', 'iid', 'rui', 'est', 'details'])
predictions_df['# of user ratings'] = predictions_df.uid.apply(get_num_user_ratings)
predictions_df['# of item ratings'] = predictions_df.iid.apply(get_num_item_ratings)
predictions_df['error'] = abs(predictions_df.est - predictions_df.rui)
best_predictions = predictions_df.sort_values(by='error')[:10]
worst_predictions = predictions_df.sort_values(by='error')[-10:]


best_predictions
```

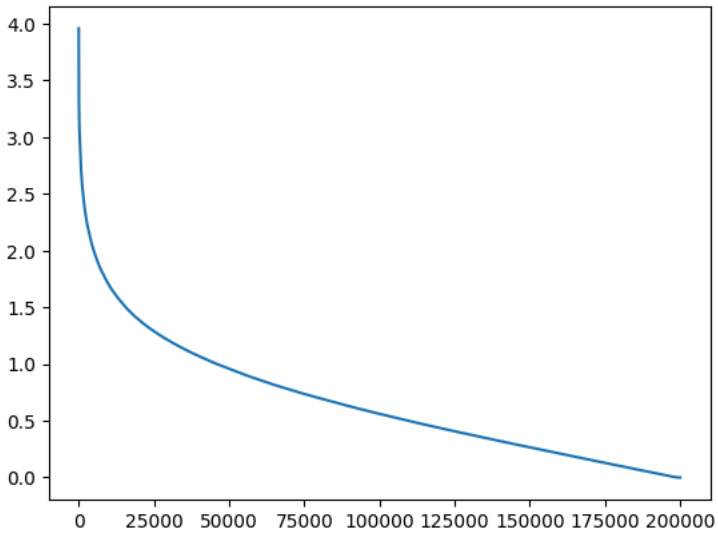
	uid	iid	rui	est	details	# of user ratings	# of item ratings	error
126763	235	904	5.0	5.0	{'was_impossible': False}	343	832	0.0
19019	3716	2571	5.0	5.0	{'was_impossible': False}	319	2043	0.0
76227	1340	173	1.0	1.0	{'was_impossible': False}	685	458	0.0
110543	3610	2648	1.0	1.0	{'was_impossible': False}	564	331	0.0
108528	1804	590	5.0	5.0	{'was_impossible': False}	93	1159	0.0
38508	1835	1285	5.0	5.0	{'was_impossible': False}	614	613	0.0

Next steps: View recommended plots

```
#Візуалізація накращих прогнозів
values = sorted(predictions2, key=lambda x: abs(x.est - x.r_ui))
y, x = [], []
for i in range(len(values)):
  x.append(abs(values[i].r_ui - values[i].est))
  y.append(i)
plt.plot(y, x)
plt.show()
```



```
# візуалізація найгірших прогнозів
values = sorted(predictions2, key=lambda x: abs(x.est - x.r_ui), reverse=True)
y, x = [], []
for i in range(len(values)):
  x.append(abs(values[i].r_ui - values[i].est))
  y.append(i)
plt.plot(y, x)
plt.show()
```




```
worst_predictions
```

	uid	iid	rui	est	details	# of user ratings	# of item ratings	error
112028	296	920	1.0	4.717903	{'was_impossible': False}	72	927	3.717903
75267	5878	1945	1.0	4.739151	{'was_impossible': False}	481	410	3.739151
121293	2673	3929	1.0	4.761863	{'was_impossible': False}	18	117	3.761863
132225	1111	1193	1.0	4.786299	{'was_impossible': False}	27	1374	3.786299
112410	68	3030	1.0	4.806648	{'was_impossible': False}	60	181	3.806648
186590	231	2858	1.0	4.828997	{'was_impossible': False}	78	2765	3.828997

Next steps: View recommended plots

Системі не вистачає людського фактору. Система діє лише математично, в той час як людина може зробити зангажлваний вибір.

```
from surprise import SVD
param_grid = {'n_epochs': [10, 20], 'lr_all': [0.001, 0.01], 'reg_all': [0.01, 0.05, 0.09], 'cache_ratings':[True]}

# метрики для оцінки ("rmse" та "mae") та кількість розбиттів для перехресної перевірки (cv=5).
# Потім ви викликаю метод fit для навчання моделі з різними комбінаціями параметрів.
gs = GridSearchCV(SVDpp, param_grid, measures=["rmse", "mae"], cv=5)
gs.fit(data)

# Після навчання моделі  вивожу найкращий результат (найменший RMSE) та параметри, які призвели до цього результату.
print(gs.best_score["rmse"])
print(gs.best_params["rmse"])


algo_gridSearch = SVDpp(**gs.best_params['rmse'])
# створюю повний навчальний набір даних. Це означає, що модель буде навчена на всьому наборі даних
trainset, testset = train_test_split(data, test_size=0.3)


# навчання вибраної моделі algo на побудованому повному навчальному наборі даних trainset.
algo_gridSearch.fit(trainset)
predictions2=algo_gridSearch.test(testset)
accuracy.rmse(predictions2)
accuracy.mae(predictions2)
```

```
def get_top_n(predictions, n=5):

    #  створюється порожній словник top_n, де ключами будуть ID користувачів, а значеннями — списки прогнозованих оцінок для
    top_n = defaultdict(list)
    # Перебирає кожен прогноз і додає його до відповідного користувача в словнику.
    for uid, iid, true_r, est, _ in predictions:
        top_n[uid].append((iid, est))

    # Сортує прогнозовані оцінки для кожного користувача та вибирає топ-N.
    for uid, user_ratings in top_n.items():
        user_ratings.sort(key=lambda x: x[1], reverse=True)
        top_n[uid] = user_ratings[:n]
    # Повертає оновлений словник з топ-N рекомендаціями для кожного користувача
    return top_n


top_n = get_top_n(predictions2)

# Виводить ID користувачів та їх рекомендації
for uid, user_ratings in top_n.items():
    print(uid, [iid for (iid, _) in user_ratings])
```

```
4004 ['2997', '2330', '2375', '2399', '3671']
595 ['318', '2858', '2762', '858', '47']
3847 ['593', '527', '1136', '1704', '1197']
479 ['260', '356', '11', '1307', '1196']
4747 ['2762', '1196', '150', '589', '1721']
956 ['913', '2571', '1947', '1233', '556']
1530 ['1213', '1201', '1196', '909', '2248']
5634 ['260', '527', '2628', '2706', '50']
3908 ['923', '1252', '3429', '3683', '2427']
4177 ['593', '589', '296', '527', '2571']
5777 ['608', '1230', '1104', '2858', '1203']
4384 ['2858', '1196', '296', '1197', '2804']
369 ['2804', '588', '2396', '3617', '1580']
5301 ['110', '1036', '2706', '1242', '1961']
5448 ['34', '1947', '1252', '1193', '951']
3620 ['318', '2028', '3578', '1954', '589']
6016 ['858', '1300', '969', '2357', '3822']
3773 ['1198', '260', '1234', '50', '3360']
3635 ['969', '2186', '1283', '2203', '2935']
197 ['593', '1090', '3481', '412', '1370']
528 ['3844', '539', '3147', '1784', '3178']
4998 ['1148', '720', '593', '1285', '2788']
1809 ['2959', '2700', '2716', '2706', '2763']
198 ['1196', '1210', '1', '953', '2804']
1111 ['1193', '2085', '932', '1608', '1592']
5938 ['2858', '923', '162', '2494', '2010']
3958 ['3114', '593', '1197', '380', '3751']
4975 ['110', '1097', '150', '1225', '2918']
4071 ['1257', '1214', '2791', '2000', '2288']
823 ['246', '1259', '3007', '1', '3504']
3576 ['593', '1291', '3421', '1226', '2019']
3579 ['1527', '1275', '2167', '1036', '1201']
2766 ['2268', '1234', '457', '1197', '364']
4344 ['858', '2905', '1252', '1237', '1228']
251 ['1259', '2918', '3361', '590', '3916']
4173 ['260', '1088', '1035', '3256', '356']
4656 ['1276', '1213', '1136', '2396', '608']
3754 ['858', '1307', '260', '1228', '3360']
2834 ['3801', '971', '948', '17', '1968']
2827 ['1231', '2707', '1127', '2770', '2717']
2597 ['969', '1246', '1296', '2080', '446']
285 ['858', '1221', '1208', '1193', '1219']
921 ['527', '3578', '47', '2324', '356']
5137 ['745', '3108', '2140', '551', '2139']
139 ['260', '1197', '720', '1527', '3052']
312 ['2019', '260', '3307', '296', '1953']
2027 ['2028', '2396', '1225', '1197', '3105']
1427 ['318', '1210', '3114', '1', '2918']
336 ['1210', '1909', '3617', '1573', '748']
2078 ['50', '1704', '2762', '150', '1197']
3525 ['608', '32', '1197', '1411', '1339']
1562 ['3081', '724', '1999', '2279', '3709']
5969 ['1028', '1234', '911', '2671', '2804']
2692 ['2571', '745', '457', '1784', '1500']
988 ['1259', '3201', '2997', '2918', '2571']
4828 ['356', '2406', '2572', '1210', '1188']
3732 ['527', '2324', '1284', '1234', '1617']
5112 ['912', '899', '1267', '1617', '3467']
4979 ['260', '1196', '1197', '720', '1266']
```

```
from surprise import Prediction
from surprise import NormalPredictor
```

```
algo_normal_pred = NormalPredictor()

# trainset, testset = train_test_split(data, test_size=0.2)

algo_normal_pred.fit(trainset)

predictions_normal = algo_normal_pred.test(testset)
rmse_score = accuracy.rmse(predictions_normal)
mae_score = accuracy.mae(predictions_normal)
```

```
RMSE: 1.5085
MAE:  1.2095
```

```
# для кожного прогнозу показані: хто (користувач), що (предмет), яка була реальна оцінка, яку модель прогнозувала. кількість
# trainset = algo_test.trainset
predictions_df_normal = pd.DataFrame(predictions_normal, columns=['uid', 'iid', 'rui', 'est', 'details'])
predictions_df_normal['error'] = abs(predictions_df_normal.est - predictions_df_normal.rui)
best_predictions_normal = predictions_df_normal.sort_values(by='error')[:10]
worst_predictions_normal = predictions_df_normal.sort_values(by='error')[-10:]
```

```
best_predictions_normal
```

	uid	iid	rui	est	details	error
0	1709	1011	5.0	5.0	{'was_impossible': False}	0.0
91625	4094	3360	5.0	5.0	{'was_impossible': False}	0.0
148722	524	628	5.0	5.0	{'was_impossible': False}	0.0
43054	2124	1299	5.0	5.0	{'was_impossible': False}	0.0
148743	2581	915	5.0	5.0	{'was_impossible': False}	0.0
43084	2073	2300	5.0	5.0	{'was_impossible': False}	0.0
43086	2952	1259	5.0	5.0	{'was_impossible': False}	0.0
91563	5011	2091	5.0	5.0	{'was_impossible': False}	0.0
175783	1311	1333	5.0	5.0	{'was_impossible': False}	0.0
43107	5102	2144	5.0	5.0	{'was_impossible': False}	0.0

Next steps: View recommended plots

```
df
```

	user_id	item_id	rating	timestamp
0	1	1193	5.0	978300760
1	1	661	3.0	978302109
2	1	914	3.0	978301968
3	1	3408	4.0	978300275
4	1	2355	5.0	978824291
...	...	...	...	...
1000204	6040	1091	1.0	956716541
1000205	6040	1094	5.0	956704887
1000206	6040	562	5.0	956704746
1000207	6040	1096	4.0	956715648
1000208	6040	1097	4.0	956715569

1000209 rows × 4 columns

```
from surprise import Prediction

ratings_data = df.groupby('item_id')['rating'].mean().to_dict()

# Створює порожній список predictions_avg, де будуть зберігатися нові прогнози зі зміненими рейтингами.
predictions_avg = []
for pred in predictions2:

    average = ratings_data.get(pred.iid)
    if average is not None:
      # Додає новий прогноз до списку predictions_avg зі зміненим рейтингом на основі середнього рейтингу жарту.
        predictions_avg.append(Prediction(uid=pred.uid, iid=pred.iid, r_ui=pred.r_ui, est=average, details={'key': 'value'})

rmse = accuracy.rmse(predictions_avg)
mae = accuracy.mae(predictions_avg)
```

```
RMSE: 0.9780
MAE:  0.7813
```

```
median_data = df.groupby('item_id')['rating'].median().to_dict()

predictions_med = []
for pred in predictions2:
    median = median_data.get(pred.iid, None)
    if median is not None:
        predictions_med.append(Prediction(uid=pred.uid, iid=pred.iid, r_ui=pred.r_ui, est=median, details={'key': 'value'}))

rmse = accuracy.rmse(predictions_med)
mae = accuracy.mae(predictions_med)
```

```
RMSE: 1.0271
MAE:  0.7401
```